

etcd — Operator Manual

What this technology does

etcd is the three-member distributed coordination store used by Patroni and other tightly scoped HA coordination logic. It is **not** the telemetry database.

A Patroni/PostgreSQL process can be running while safe leader election is impossible if etcd quorum is lost.

Current commissioned topology

Host	Member	Client URL	Peer URL
ULC-01	etcd-01	http://10.104.0.2:2379	http://10.104.0.2:2380
ULC-02	etcd-02	http://10.104.0.4:2379	http://10.104.0.4:2380
ULC-03	etcd-03	http://10.104.0.8:2379	http://10.104.0.8:2380

The commissioned etcd transport is private-network HTTP. Do not invent TLS flags until an etcd TLS migration is actually deployed.

Current Compose layout is under `/opt/lorawan/etcd`; persistent data is under `/opt/lorawan/etcd/data`.

Step 1 — Check container state on all three nodes

Run on each ULC host:

```
cd /opt/lorawan/etcd
sudo docker compose ps
```

PASS means: the intended etcd container is running with no restart loop.

One running member is not quorum proof. Continue.

Step 2 — Check all endpoint health from one approved node

```
sudo docker exec etcd etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health
```

PASS means: all three endpoints report healthy in normal state.

During an approved single-member failure, two healthy voting members can still provide quorum, but restore 3/3 before another fault.

Step 3 — Check endpoint status and leader

```
sudo docker exec etcd etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint status --write-out=table
```

Check:

- exactly one current Raft leader;
- all intended members reachable;
- no obviously divergent/lagging revision;
- no unexplained errors.

The leader can legitimately change. Do not force it back to a node just to match an old screenshot.

Step 4 — Verify member list

```
sudo docker exec etcd etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  member list --write-out=table
```

PASS means: the expected three members exist, with the correct peer URLs. There should be no unexplained stale/learner member.

Step 5 — Verify network/listener exposure

On each host:

```
sudo ss -lntp | grep -E ':(2379|2380)\b'
```

The intended etcd service must remain on the private east-west addresses/loopback defined by the deployment. Do not expose etcd on 0.0.0.0 or a public interface to simplify troubleshooting.

Step 6 — Verify Patroni sees a stable DCS before database actions

On a PostgreSQL host, use the Patroni guide to verify one leader and the expected members. If Patroni leadership is ambiguous, return to etcd health before attempting switchover/promotion.

Never force-promote PostgreSQL to work around etcd quorum loss.

Safe maintenance rule

For planned work on one etcd member:

1. prove current 3/3 health;
2. prove which member/host you are touching;
3. ensure the other two are healthy and mutually reachable;
4. stop/maintain only one member;
5. verify surviving quorum before further work;

6. restore the member;
7. verify 3/3 endpoint health/status/member list;
8. verify Patroni DCS/leader stability.

Do not maintain two voting members at once.

Snapshot/backup rule

Before destructive etcd recovery or cluster-wide changes, create and verify the snapshot using the recovery runbook. A file existing on disk is not enough; record its creation time, size/hash, and restore procedure.

Use [../server/cloud-production/13-backup-restore-and-disaster-recovery.md](#) for the approved backup/recovery boundary.

Troubleshooting order

Symptom	Inspect first
One member down, two healthy	failed host/container/network; preserve quorum
No leader / writes fail cluster-wide	network partition, two-member loss, disk/full/corruption
Member cannot join peers	10.104.0.x:2380 reachability and member URL/config
Client cannot query one node	2379, local/container state, bind address
Patroni no leader despite PostgreSQL running	etcd quorum/DCS first

Useful bounded logs on a member:

```
cd /opt/lorawan/etcd
sudo docker compose logs --since=15m --tail=150 etcd
```

Recovery hard stop

Do **not** delete /opt/lorawan/etcd/data, start a member with initial-cluster-state: new, remove/re-add members, or restore a snapshot into the live cluster because one health check failed. Those are recovery operations and can destroy quorum/state when used casually.

Completion checklist

- three intended members present;
- normal state endpoint health is 3/3;
- exactly one Raft leader;
- private listener exposure only;
- Patroni DCS state is stable;
- no unexplained stale member;
- current backup/recovery path is known before destructive work.

Detailed runbooks

- [../server/cloud-production/05-etcd-cluster.md](#)
- [../server/cloud-production/13-backup-restore-and-disaster-recovery.md](#)

Supporting runbook: 13-backup-restore-and-disaster-recovery.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

13. Minimal POC Backup and Recovery

Status: 13A FAST-PATH PASS / 13S SERVER-ONLY SNAPSHOT COMPLETE + LOCALLY VERIFIED / 13B FINAL LATER. Phase 13A protects the pre-cutover cloud state. Interim **13S server-only snapshot/export** has already captured and locally verified the commissioned server baseline, including the protected PostgreSQL/etcd/OpenBao and reconstruction artifacts described below. **13S is not the final Phase 13B backup set.** Phase 13B must still be recreated after every required pre-test service and real application path are commissioned. Off-host copy and isolated restore proof remain deferred to the pre-Phase-15 destructive/recovery boundary; intentional destructive recovery tests belong to Phase 15.

This HA POC is **not** trying to prove a production disaster-recovery platform. It still needs a real rollback boundary before we deliberately kill leaders, members, and hosts.

HA-preserving scope boundary: Phase 13 does not authorize removing or bypassing any commissioned HA component. Patroni/Spilo, etcd, HAProxy, PgBouncer, Valkey/Sentinel, MQTT redundancy, and the two-node ChirpStack design remain part of the active system. For normal functional commissioning, use the already-passed live HA health/routing evidence plus the fresh validated local logical dumps created on 2026-08-27. The more rigorous off-Droplet copy and isolated-restore rehearsal may be deferred until the dedicated destructive/failure-injection boundary unless a new failure signal, migration risk, or operator requirement makes them immediately necessary. Do not confuse "defer rigorous DR proof" with "remove HA".

The rule is simple:

Do not inject a destructive failure until the fully commissioned state needed to rebuild the POC exists outside the three target Droplets and the required isolated restore checks have passed.

13.0 Two mandatory backup checkpoints

Phase 13A - before gateway/device cutover

Normally complete after Phase 11 and before Phase 12. If the physical gateway is temporarily unavailable, the **cloud-only 13A checkpoint may be executed and closed first** because it does not mutate gateway state: protect the commissioned cloud databases/configuration, copy the backups off the target Droplets, validate catalogs/checksums, and complete an isolated `lorawan_telemetry` restore with Timescale objects intact. When a legacy authoritative source exists, protect that source independently as part of the migration plan. Phase 12 still requires both Phase 11 normal-path commissioning and Phase 13A PASS; finishing 13A early does not waive the gateway prerequisite.

13A PASS permits Phase 12 cutover/provisioning. It does not permit Phase 15 fault injection.

Phase 13S - interim server-only snapshot/export while external dependencies are deferred

13S exists only to remove avoidable server-side work from the final commissioning day. It may run now against the commissioned server components and should capture non-secret reconstruction material for etcd, PostgreSQL/Patroni/TimescaleDB, HAProxy/PgBouncer, Mosquitto, Valkey/Sentinel, ChirpStack, Node-RED, Grafana, OpenBao, and the Fabric outbox database layer. It may also record current image digests, configuration hashes, service/listener state, and a fresh logical database backup if the operator deliberately wants a newer rollback point.

Do **not** fabricate missing artifacts. The original 13S pre-staging pass predated later commissioning of the physical gateway normal path, real EMU-01 rows, the HRC Task 37 handoff, ULC-01 Fabric production writes, adapter images/configuration, and gateway-integrity runtimes; those artifacts now exist and must be included in the next refreshed recovery checkpoint from their verified current locations. The genuinely open infrastructure gates are provider Cloud Firewall/control-plane evidence and Reserved-IP reassignment/failover acceptance, plus ULC-02 Fabric ownership/fencing acceptance and any explicitly deferred destructive restore/failover tests. Do not place plaintext passwords, private keys, AppKeys, OpenBao root/recovery material, Fabric private credentials, or bearer tokens in the general 13S archive.

For normal server preparation, a successful 13S checkpoint means the locally verified server-only package is intact and reproducible. Once `PHASE13S_LOCAL_PACKAGE=PASS` has been reached, treat `SERVER_ONLY_SNAPSHOT_EXPORT=PASS` as satisfied for the current non-destructive commissioning scope and stop repeating backup/export work. Off-host copying, isolated restore rehearsal, and stronger disaster-recovery proof are deferred to the dedicated destructive/failure-testing boundary before Phase 15. This still does **not** satisfy final Phase 13B, which must be refreshed after the remaining provider, hardware, Fabric, and final acceptance work is commissioned.

Phase 13S-1 initial three-host preflight harness stop - 2026-08-29

The first 13S-1 wrapper proved the `ulc-03` control-host/free-space gate and noninteractive SSH + `sudo -n` access to both `ulc-01` and `ulc-02`, then returned cleanly to the `ulc-03` prompt before building or running the host probe. No server/service/database mutation occurred. The stop is a shell-transport harness defect: the wrapper itself was being supplied to `sudo bash` through a heredoc, while the Step 2 `ssh` commands inherited that same stdin. SSH is allowed to consume stdin, so it consumed the remaining wrapper text after the access checks. This is not evidence of a server, SSH, `sudo`, backup-tool, or service failure.

Do not repeat the already-passed access gates merely to diagnose this. Continue from probe construction. When an SSH command inside a heredoc-driven wrapper does **not** intentionally transport a script or payload over stdin, use `ssh -n` or redirect stdin from `/dev/null`. When stdin transport is intentional, redirect from the exact file being sent (for example `ssh ... 'sudo -n bash -s -- ulc-01' < "$PROBE"`) so SSH cannot consume the parent wrapper.

Phase 13S-1 resumed inventory partial PASS / listener-filter harness defect - 2026-08-29

The corrected resume reached all three hosts and proved the useful inventory through OpenBao tooling. `ulc-01`, `ulc-02`, and `ulc-03` all reported Ubuntu 24.04.4 LTS, zero failed systemd units, healthy commissioned service/container baselines, `etcdctl 3.4.30`, PostgreSQL 18.6 `pg_dump/pg_restore`, and OpenBao CLI 2.6.2. Container restart counts were zero and no OOM state was reported. `ulc-03` additionally showed Grafana and active Node-RED running as expected. Patroni remained exactly one leader on `10.104.0.2` plus two replicas on `.4` and `.8`.

The probe then hit `awk`: unexpected newline or end of string in the multi-line listener filter on every host. Because the outer resume wrapper did not use fail-fast handling for each remote probe, it continued to Patroni and printed a final `PHASE13S_SERVER_INVENTORY_PREFLIGHT=PASS` banner even though the per-host probe had exited before listener output, non-secret configuration metadata, and `HOST_PREFLIGHT_*` markers. Therefore that final

banner was not authoritative until the missing tail was rerun. Preserve all successful inventory/tool/resource/Patroni evidence and do not repeat OS/resource/tool discovery.

Phase 13S-1 three-host inventory preflight COMPLETE / PASS - 2026-08-29

The corrected listener/configuration tail passed on all three servers. Every host retained the required etcd, PostgreSQL, PgBouncer, Patroni, OpenBao, and HAProxy database listeners. u1c-01 and u1c-02 also retained the commissioned MQTT, ChirpStack, and OpenBao HAProxy listeners; u1c-03 retained Node-RED on 127.0.0.1:1880, Grafana on 127.0.0.1:3000, and the local Node-RED MQTT frontend on 10.104.0.8:18884. Non-secret reconstruction configuration metadata and SHA-256 values were captured without displaying contents, while PgBouncer userlists and Node-RED/Grafana protected environment files were represented by metadata only.

Combined with the immediately preceding inventory, all three hosts have zero failed units, available backup tooling (etcdctl 3.4.30, PostgreSQL 18.6 pg_dump/pg_restore, OpenBao CLI 2.6.2), current containers with restart count zero/OOM false, and the expected commissioned service layout. Patroni remains one leader on 10.104.0.2 with replicas on .4 and .8. HOST_PREFLIGHT_u1c-01=PASS, HOST_PREFLIGHT_u1c-02=PASS, HOST_PREFLIGHT_u1c-03=PASS, PATRONI_BASELINE=PASS, and PHASE13S_SERVER_INVENTORY_PREFLIGHT=PASS are authoritative. No service or database state changed.

Next: create a protected timestamped 13S snapshot set containing fresh custom-format chirpstack and lorawan_telemetry logical dumps plus a validated etcd v3 snapshot/member record. OpenBao Raft snapshotting is a separate privileged sub-boundary; never use the Fabric adapter identity for operator snapshot privileges.

Phase 13S-2 PostgreSQL + etcd snapshot COMPLETE / PASS - 2026-08-29

The protected server-only snapshot set at /home/opsadmin/backups/phase13s-20260829T022146Z is valid. Fresh custom-format dumps were created from the healthy u1c-03 Spilo replica: chirpstack.dump is 101218 bytes and lorawan_telemetry.dump is 72384 bytes; both passed pg_restore --list. The repeated en_US.utf-8 Perl warnings remain non-blocking locale noise, and the TimescaleDB continuous_agg circular-foreign-key warning is the same known full custom-dump warning already recorded in Phase 13A.

The first etcd health-count attempt stopped only because etcdctl endpoint health emitted the three successful endpoint results on stderr while the wrapper redirected stdout only. No etcd failure occurred and the PostgreSQL dumps were preserved rather than recreated. The corrected resume captured stdout+stderr for each endpoint and proved all three endpoints healthy. etcdctl is 3.4.30; the live etcd servers are 3.5.15. The authoritative member/status record showed three started non-learner members, etcd-02 on 10.104.0.4 as current etcd leader, and a common Raft term 2 / index 1387 at capture time.

A v3 snapshot was then created from 10.104.0.8:2379 as etcd.snapshot, size 491552 bytes. Snapshot status reported hash c0b2b3a5, revision 1326, 1336 total keys, and about 492 kB. The snapshot set contains nine mode-0600 opsadmin:opsadmin files under a mode-0700 opsadmin:opsadmin directory: source metadata, both database dumps, etcd health/status/member evidence, etcd snapshot/status, and SHA256SUMS. sha256sum -c passed before and after ownership/permission normalization. Patroni still reported exactly one leader on 10.104.0.2 and replicas on .4/.8 after snapshot creation. PHASE13S_POSTGRES_ETCD_SNAPSHOT=PASS is authoritative. Do not recreate these exact dumps/snapshot unless the source state changes; continue by adding secret-free reconstruction/application exports in a separate 13S sub-boundary.

Phase 13S-2 PostgreSQL dumps PASS / etcd health-capture harness stop - 2026-08-29

The protected snapshot directory /home/opsadmin/backups/phase13s-20260829T022146Z was created successfully on u1c-03. The pre-snapshot gate proved u1c-03 is still a Patroni replica with replay caught up (true|true), etcdctl is 3.4.30, and sufficient backup space exists. SOURCE-METADATA.txt was created, a fresh chirpstack.dump of 101218 bytes passed pg_restore --list, and a fresh lorawan_telemetry.dump of 72384 bytes passed pg_restore --list. The repeated en_US.utf-8 locale warning remains non-blocking. The TimescaleDB

continuous_agg circular-foreign-key warning is the already-known full custom-format dump warning and did not invalidate the telemetry catalog check.

Step 6 then stopped on a **health-output capture defect, not an etcd health failure**. etcdctl endpoint health visibly returned successful committed-proposal health for all three endpoints (10.104.0.2, .4, .8), but this etcdctl build emitted those human-readable health lines on stderr. The wrapper redirected only stdout to etcd-endpoint-health.txt, leaving that file empty; the subsequent grep -c 'is healthy' therefore returned zero and intentionally stopped before snapshot creation. Do not recreate the two valid PostgreSQL dumps. Reuse this exact partial directory, capture each etcd endpoint health with stderr included (or rely on the command exit status), then continue with endpoint/member records, etcd snapshot, SHA-256 manifest, ownership/mode normalization, and the final Patroni regression gate.

Phase 13S-3 secret-free reconstruction export COMPLETE / PASS - 2026-08-29

The reconstruction bundle under /home/opsadmin/backups/phase13s-20260829T022146Z/reconstruction is complete for the currently commissioned server-only state. Three host manifests capture OS/service/container/listener state plus safe configuration hashes and protected-file metadata without copying environment files, private keys, PgBouncer userlists, or credential databases. Node-RED B was re-proved fenced with no :1880 listener and no flows_cred.json, then the exact reviewed shared runtime (compose.yml, settings.js, package.json, package-lock.json, flows.json) was exported from the stopped standby and matched all authoritative hashes. Grafana compose.yml, datasource provisioning, dashboard provisioning, and lorawan-overview.json passed literal-secret checks before export; grafana.env remained excluded. The reconstruction tree contains 16 mode-0600 opsadmin:opsadmin files under mode-0700 directories and its own SHA-256 manifest passed.

The original Phase 13S-2 PostgreSQL/etcd SHA256SUMS was rechecked afterward and remained unchanged. Node-RED A remained running|0|healthy, Grafana running|0|false, and Node-RED B remained fenced.

PHASE13S_RECONSTRUCTION_EXPORT=PASS is authoritative. Provider-owned public ingress, physical gateway/EMU-01 acceptance, Fabric adapter/handoff, gateway-integrity runtime, final Phase 13B, Phase 14, and Phase 15 remain explicitly deferred or blocked as recorded in DEFERRED-AND-SEPARATE.txt.

The next server-only backup boundary is OpenBao. Cloud production currently has no dedicated snapshot-only backup AppRole. Create and verify a least-privilege openbao-raft-snapshot-reader policy and openbao-backup AppRole before taking the Raft snapshot. Do not use the fabric-adapter role and do not take the snapshot with a long-lived root token. Prefer short-lived one-use SecretIDs that are generated only for the snapshot operation and never stored in the general reconstruction archive.

Phase 13S-4A OpenBao snapshot-only backup identity COMPLETE / PASS - 2026-08-29

The three-node OpenBao cluster was healthy and unsealed before mutation. The existing lorawan-evidence Transit key remained ecdsa-p256, non-exportable, plaintext backup disabled, deletion disabled, and version 1; the fabric-adapter AppRole retained only the fabric-evidence-signer policy and had zero SecretID accessors.

A new policy openbao-raft-snapshot-reader was created with exactly one path: sys/storage/raft/snapshot, capabilities read and sudo. A new openbao-backup AppRole was then created with only that policy, token TTL 900 seconds, token max TTL 1800 seconds, SecretID TTL 900 seconds, secret_id_num_uses=1, and bind_secret_id=true. The RoleID exists but was not printed. No backup SecretID was issued during this step and the Fabric adapter still had zero SecretID accessors afterward. All three OpenBao nodes remained initialized and unsealed after the mutation. PHASE13S_OPENBAO_BACKUP_IDENTITY=PASS is authoritative.

Next: issue exactly one ephemeral backup SecretID in memory, authenticate once, prove a second login with the same SecretID fails, verify the resulting token has read,sudo only on sys/storage/raft/snapshot and deny on Transit sign/verify and Raft restore/force paths, save one OpenBao Raft snapshot, transfer it into the protected Phase 13S set, revoke the short-lived token, and confirm no reusable backup SecretID remains. Do not use the root token for the snapshot operation itself.

Phase 13S-4B first snapshot attempt stopped after SecretID issuance - 2026-08-29

The protected OpenBao snapshot workflow passed the existing 13S integrity check, loaded the protected root token without printing it, reverified the openbao-backup role, proved zero backup SecretID accessors before issuance, and confirmed a three-voter Raft configuration with 10.104.0.2 as leader at that checkpoint. The root administrative identity then successfully issued one new one-use backup SecretID. Immediately afterward, the wrapper attempted to count SecretID accessors using a Python parser that assumed the JSON document was an object containing data.keys. OpenBao 2.6.2 returned the list result as a JSON list in this invocation, so Python raised `AttributeError: 'list' object has no attribute 'get'` and the remote shell exited before AppRole login or snapshot creation.

Treat this as a harness/schema-shape defect, not an OpenBao or Raft failure. The just-issued SecretID value existed only in the failed shell and is no longer recoverable, but its server-side accessor may remain until its 15-minute TTL expires because it was never consumed. Before issuing another SecretID, list accessors with a parser that accepts both top-level JSON lists and wrapped data.keys forms. Require at most one accessor because the role had zero immediately before the failed issuance; if one orphan accessor remains, destroy that exact accessor administratively and re-prove zero. Preserve any partial local openbao-raft-evidence.txt transcript separately. Do not rerun Phase 13S-4A, do not recreate the backup role/policy, and do not claim an OpenBao Raft snapshot exists from this failed run.

Phase 13S-4B OpenBao Raft snapshot COMPLETE / PASS - 2026-08-29

The corrected resume found exactly one orphan openbao-backup SecretID accessor from the failed parser run, destroyed that accessor administratively, and re-proved a zero-accessor clean baseline before minting a fresh credential. The fresh SecretID was issued under the commissioned 15m / one-use backup-role constraints, authenticated successfully exactly once, and a second login with the same SecretID was rejected. The consumed accessor disappeared immediately.

The resulting backup token was verified to have exactly read + sudo on sys/storage/raft/snapshot, while Transit sign, Transit verify, and Raft restore/force capabilities were all denied. Using this restricted token against the current Raft leader 10.104.0.2, OpenBao created /tmp/phase13s-openbao.snap; the protected host copy was 30236 bytes with SHA-256 9965f9e904b83d62b07ebb9321af1b4a45a7f2b15fbfd50d0c7238fac249e68. The token was then explicitly revoked. Final backup SecretID accessors = 0; Fabric adapter SecretID accessors = 0; all three OpenBao members remained initialized/unsealed.

The snapshot was transferred into /home/opsadmin/backups/phase13s-20260829T022146Z/openbao.snap, and its destination SHA-256 exactly matched the source. The remote staging copy was removed. openbao-raft-evidence.txt, the preserved failed-run transcript openbao-raft-evidence.partial-accessor-parser.txt, and OPENBAO-SHA256SUMS were protected as mode 0600 opsadmin:opsadmin; the OpenBao manifest verified all three artifacts. The original PostgreSQL/etcd snapshot manifest and the reconstruction manifest still pass unchanged. PHASE13S_OPENBAO_RAFT_SNAPSHOT=PASS is authoritative.

Phase 13S-5 final local server-only package COMPLETE / PASS - 2026-08-29

The complete interim server-only snapshot set under /home/opsadmin/backups/phase13s-20260829T022146Z was revalidated before packaging. The existing PostgreSQL/etcd, OpenBao, and reconstruction manifests all passed. Required database dumps, etcd snapshot, OpenBao Raft snapshot, Node-RED reviewed runtime export, Grafana provisioning/dashboard export, and deferred/blocker record were present. A filename-level secret gate found no .env, private-key, password/userlist, SecretID, root-token, recovery-share, or unseal-share files in the general archive. All source files remained mode 0600 opsadmin:opsadmin and directories mode 0700 opsadmin:opsadmin.

SERVER-ONLY-STATUS.txt records the server-only/interim scope and explicitly leaves provider ingress, physical gateway/real EMU-01 acceptance, Fabric runtime/handoff/transaction, gateway-integrity runtime, final Phase 13B, final Phase 14, Phase 14B, and Phase 15 unclaimed. PHASE13S-FULL-SHA256SUMS covers 30 payload files and passed. The transport archive /home/opsadmin/backups/phase13s-20260829T022146Z.tar.gz was created outside

the source directory, is 108819 bytes, passed `gzip -t`, contained all required top-level artifacts, and passed a disposable extraction followed by `sha256sum -c PHASE13S-FULL-SHA256SUMS`. Its transport SHA-256 is 19f2072fd4ef41e34442c7cc3949decd62a0cb2cee155bcf734f24121397cc; the sidecar is /home/opsadmin/backups/phase13s-20260829T022146Z.tar.gz.sha256. PHASE13S_LOCAL_PACKAGE=PASS is authoritative.

Streamlined commissioning decision - 2026-08-29: the locally verified package is sufficient for the current server-first preparation scope. The archive has already passed its full 30-file SHA-256 manifest, gzip structure test, disposable extraction, and extracted-manifest verification, so `SERVER_ONLY_SNAPSHOT_EXPORT=PASS` is accepted now. Do not copy it off-host merely to continue normal-path setup, and do not rebuild or rehash it again unless the protected source state materially changes. The `SERVER-ONLY-STATUS.txt` inside this already-built archive still contains the earlier stricter `NOT_YET_OFFHOST_VERIFIED` wording; preserve that file as historical evidence rather than rebuilding the archive solely to change a label. Off-host copy and isolated restore proof remain deferred to the pre-Phase-15 destructive/recovery boundary. This is still an interim Phase 13S package, **not** final Phase 13B.

Phase 13B - after every pre-test service is commissioned

Re-run the backup collection after Phase 12A, Phase 14A, and Phase 20. Include Node-RED flow/config, Grafana dashboard/data-source exports, OpenBao recovery/Raft snapshot material, Fabric adapter image/config references, public-ingress scripts/units, and the final database state.

13B PASS is one prerequisite of Phase 14B. The backup identifier used by Phase 15 must refer to this fully commissioned snapshot, not an earlier 13S/13A partial build.

13.1 What to protect

Before destructive tests keep copies of:

```
chirpstack logical database dump
lorawan_telemetry logical database dump
etcd cluster snapshot + member/config record
OpenBao recovery/unseal material outside the runtime hosts
OpenBao Raft snapshot before destructive KMS recovery tests
HAProxy/PgBouncer configs
public-ingress failover scripts + systemd units/timer
Reserved IPv4 + ha-01/ha-02 Droplet IDs + anchor IPv4 record
ChirpStack configs/region files
Mosquitto configs/ACLs
Valkey/Sentinel configs
Node-RED flow export
Grafana dashboard export
Fabric handoff metadata
Fabric adapter config/image reference when an implementation exists
```

Do not create paid object storage only because a future production design normally would. For this POC, the administration workstation or another protected system outside ha-01/02/03 is sufficient as the off-host copy.

Current Phase 13A preflight result - 2026-08-27

The cloud-only read-only preflight is **PASS**. The initial checker falsely treated missing ulc-03 :16379 and :18883 listeners as failures. Those frontends are intentionally application-node-only: Valkey writable-primary HAProxy is commissioned on ulc-01/02, and the ChirpStack workload MQTT HAProxy frontends are commissioned on ulc-01/02. Do not deploy either frontend on ulc-03. Patroni, etcd, all required database routes, both ChirpStack application nodes, PostgreSQL/TimescaleDB state, telemetry hypertables/schema state, backup tools, and local backup capacity passed. Continue with fresh Phase 13A dumps.

The first fresh-dump wrapper then stopped safely before creating either dump because it required `host(inet_server_addr()) = 10.104.0.8` while `psql` was using the local Unix-domain socket inside the Spilo container. On a Unix-socket PostgreSQL session `inet_server_addr()` is `NULL`, so the observed `|t` means only `pg_is_in_recovery() = true`; it is not evidence of a wrong node. For local-container backup work, verify the host identity outside SQL and require only replica state from PostgreSQL, or explicitly connect by TCP if the server address itself is part of the assertion. Reuse the existing timestamped directory only if it contains no dump artifacts; otherwise create a new timestamped directory.

13.2 Create one protected backup directory

Run on the host from which backups are being collected:

```
umask 077
BACKUP_DIR="$HOME/backups/lorawan-ha-poc-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$BACKUP_DIR"
printf '%s\n' "$BACKUP_DIR"
```

Store the test/run identifier beside the backup so later evidence shows which failure rehearsal it protected.

Do not put database passwords, private keys, or OpenBao recovery material in the general command transcript.

13.3 PostgreSQL logical dumps through the stable HA path

Both logical databases live on the same Patroni cluster:

```
chirpstack
lorawan_telemetry [TimescaleDB enabled]
```

Use a protected backup/admin role that can read the required database objects. Connect through the local PgBouncer/HAProxy route or an explicitly verified current-primary admin route.

If using PgBouncer, first confirm `<BACKUP_ROLE>` is actually admitted by the configured `auth_file/auth_query` and can connect to **both** logical databases. Do not weaken PgBouncer authentication just to make a backup work. If the backup role is intentionally excluded from PgBouncer, use a separately approved TLS-verified current-primary administration path and record that exception.

Example using the stable pool path:

```
pg_dump \
--host=pgbouncer.internal.lorawan.com \
--port=6432 \
--username=<BACKUP_ROLE> \
--dbname=chirpstack \
--format=custom \
--file="$BACKUP_DIR/chirpstack.dump"

pg_dump \
--host=pgbouncer.internal.lorawan.com \
--port=6432 \
--username=<BACKUP_ROLE> \
--dbname=lorawan_telemetry \
--format=custom \
--file="$BACKUP_DIR/lorawan_telemetry.dump"
```

Provide authentication through the approved protected mechanism such as `.pgpass`, a short-lived credential source, or an interactive prompt. Do not put the live password in the command line.

Immediately validate both catalogs:

```
pg_restore --list "$BACKUP_DIR/chirpstack.dump" >/dev/null
pg_restore --list "$BACKUP_DIR/lorawan_telemetry.dump" >/dev/null
sha256sum "$BACKUP_DIR/chirpstack.dump" "$BACKUP_DIR/lorawan_telemetry.dump" \
| tee "$BACKUP_DIR/SHA256SUMS"
```

Stop here if either `pg_restore --list` fails or either dump is zero bytes.

Why both dumps matter: `lorawan_telemetry` contains the Timescale hypertables and the current application schema. `telemetry.fabric_outbox` is included **only after it has actually been commissioned** by the later Node-RED/Fabric setup. At the current pre-Phase-12A 13A checkpoint, preserve the live source state exactly; do not create an outbox merely to make the backup match the future target architecture.

Current Phase 13A fresh logical backup result - 2026-08-27

The corrected fresh-backup retry is **PASS** on `ulc-03`. Host identity was verified outside PostgreSQL and the local Spilo member returned `pg_is_in_recovery() = true`. Both `chirpstack` and `lorawan_telemetry` exist. Source metadata recorded PostgreSQL 18.6, ChirpStack counts `tenant=1, application=0, device_profile=0, device=0, gateway=0`, telemetry counts `uplinks=0, measurements=0, device_registry=0`, TimescaleDB 2.29.2, hypertables `measurements,uplinks, and fabric_outbox_count=0`. Fresh custom-format dumps were created under `/home/opsadmin/backups/phase13a-20260827T032756Z`: `chirpstack.dump` about 99 KiB and `lorawan_telemetry.dump` about 58 KiB. Both archives passed `pg_restore --list`; `SOURCE-METADATA.txt`, both dumps, and `SHA256SUMS` are mode `0600`; `sha256sum -c SHA256SUMS` passed for all three hashed files. The TimescaleDB `continuous_agg` circular-foreign-key warning is expected for this full custom-format dump and does not invalidate the archive; the isolated restore rehearsal remains the authoritative restore proof. The database-backup sub-gate is therefore complete locally, but Phase 13A remains open until this directory is copied off the target Droplets and destination hashes are verified.

Streamlined Phase 13A path for normal-path commissioning - 2026-08-28

For the current POC, use the minimum rollback boundary needed to continue **non-destructive** Phase 12 / normal-path testing quickly. The fresh `chirpstack` and `lorawan_telemetry` dumps have already passed local `pg_restore --list` and SHA-256 verification on `ulc-03`; do not recreate or re-parse them unless the source state changes.

The fast Phase 13A checkpoint is:

```
existing verified logical dumps on ulc-03
-> package/copy once to the administration workstation
-> verify one transport SHA-256
-> retain the off-host copy
-> continue normal-path commissioning
```

Do **not** require an isolated database restore, etcd snapshot restore rehearsal, destructive membership exercise, or failure injection merely to continue normal-path commissioning. Those stronger DR proofs become mandatory before Phase 15 destructive/failover testing. This streamlining does not remove Patroni, etcd, HAProxy, PgBouncer, Valkey/Sentinel, MQTT HA, or any other HA component.

For the current backup `/home/opsadmin/backups/phase13a-20260827T032756Z`, one compressed archive plus one SHA-256 sidecar is sufficient for the transport step. The internal dump catalogs and hashes are already proven at source.

13.4 Copy PostgreSQL backups off the three Droplets

If the backup was created on ha-01, ha-02, or ha-03, copy the entire protected backup directory to the administration workstation or another approved off-host location before the destructive test.

Then recalculate the checksums at the destination and compare with SHA256SUMS.

A dump that exists only on a Droplet scheduled for failure is not a useful rollback copy.

Phase 13A off-host transport PASS - 2026-08-28

The compressed transport archive phase13a-20260827T032756Z.tar.gz was copied from u1c-03 to the Windows administration workstation using the proven id_ed25519_home_ops SSH identity. The source SHA-256 was e97d50c31252ede1fe55b734b6686f270e92ebecb69a36d637b04fbf726cda1c; Windows Get-FileHash -Algorithm SHA256 returned the same value and the operator obtained PHASE13A_OFFHOST_COPY=PASS. Treat the streamlined Phase 13A transport checkpoint as PASS. For normal-path commissioning, do not repeat the database dumps, dump-catalog checks, isolated restore rehearsal, or destructive DR tests now; the stronger recovery proof remains deferred to the later destructive/failover test boundary.

13.5 etcd snapshot before destructive coordination tests

For a normal single-member loss, the preferred recovery is to restore/rejoin the missing member using the normal etcd member procedure. A cluster snapshot is still mandatory safety evidence before deliberately changing membership or performing a destructive quorum-recovery test.

Using a compatible etcdctl against the currently tested east-west endpoints:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379 \
  snapshot save "$BACKUP_DIR/etcd.snapshot"
```

These commands match the current HTTP-only etcd deployment on 10.104.0.0/20. If etcd transport TLS is deployed later, update this procedure and use the tested CA/client credentials; do not mix the two transport modes.

Inspect the snapshot using the status command supported by the pinned etcd release (etcdctl snapshot status on releases that use etcdctl). Record:

```
etcd member list
cluster ID
member IDs / names / peer URLs
snapshot hash/revision/status
```

The etcd snapshot contains coordination state, not PostgreSQL rows.

13.6 OpenBao recovery material and Raft snapshot

Keep Shamir/recovery/unseal material **outside** ha-01/02/03 and outside the general configuration archive.

Before an OpenBao failure rehearsal record:

```
member list
active node
sealed/unsealed state
Raft health
Transit mount
lorawan-evidence key metadata
```

Before a test that can damage Raft state, use a privileged backup identity—not the Fabric adapter—to create the approved snapshot, for example with the pinned CLI:

```
bao operator raft snapshot save "$BACKUP_DIR/openbao.snap"
```

Then copy that snapshot off the three Droplets and retain the protected recovery material needed to restore/unseal the cluster.

The Fabric adapter must not receive root, recovery, or snapshot privileges.

13.7 Configuration snapshot

Capture the non-secret configuration that would be needed to reconstruct the scale model:

```
HAProxy
PgBouncer
public-ingress scripts and systemd service/timer units
Reserved IPv4, ha-01/ha-02 Droplet IDs, anchor IPv4s, and expected initial/current owner
public-ingress health/takeover/evaluator script hashes
Patroni/Spilo non-secret config and image digest
etcd member/config and pinned version
Mosquitto config + ACL + image digest
Valkey/Sentinel config + image digest
ChirpStack region/config files + image digest
Node-RED flow export + palette versions
Grafana dashboard/data-source export
OpenBao server config + version
Fabric handoff metadata
Fabric adapter non-secret config/image digest when available
```

Keep private keys/passwords in their designated protected recovery location, not in a general tar archive. In particular, do **not** copy the live DIGITALOCEAN_TOKEN from /etc/lorawan-cloud/public-ingress.env into the ordinary configuration bundle. Record its secret-store reference and required permission scope, then restore the token separately through the approved secret workflow.

13.8 Isolated PostgreSQL restore rehearsal

Do this before the first destructive HA test and after any PostgreSQL-major/TimescaleDB change.

Use a temporary target that has:

```
same compatible PostgreSQL major
same required TimescaleDB extension build available
no production data
no connection from live ChirpStack/Node-RED/Fabric adapters
```

Procedure:

1. create a temporary empty PostgreSQL instance/database outside the live three-node cluster when possible;
2. verify the TimescaleDB extension files are available for the selected PostgreSQL major;

3. create/enable the extension as required by the restore procedure;
4. run `pg_restore --list lorawan_telemetry.dump` again on the restore host;
5. restore with `--exit-on-error`;
6. query `pg_extension` and confirm `timescaledb` is present;
7. query `timescaledb_information.hypertables` and confirm `telemetry.uplinks` and `telemetry.measurements` are hypertables;
8. compare `telemetry.fabric_outbox` with the recorded source state: if the source backup contains it, confirm it restores as an ordinary table; if the source records it as `not-yet-commissioned`, confirm it remains absent rather than creating it during restore;
9. compare the small POC row counts with the source evidence;
10. verify the `telemetry_reader/writer` ownership/grants needed by the restored schema;
11. capture the sanitized restore result;
12. destroy the temporary restore target after the evidence is retained.

Do not call a backup tested merely because `pg_dump` exited zero. The restore rehearsal is what proves the dump is usable.

13.9 Outbox recovery rule

After restoring `lorawan_telemetry`:

1. keep both Fabric adapters stopped during database restore
2. validate the restored outbox and current ownership/fencing state
3. start only ULC-01 adapter-1 as the commissioned writer
4. inspect pending / processing / reconciling / confirmed / failed / needs_attention rows; treat submitted_unknown as legacy compatibility state
5. release or wait out only leases proven stale
6. reconcile any row with durable Fabric transaction material by commit status plus source-bound query/digest verification
7. verify no row is falsely marked confirmed
8. keep ULC-02 adapter-2 write-disabled unless the separate HA takeover gate is deliberately being executed

Why: a Fabric transaction may have committed even when the client timed out. Blind replay can create duplicate/conflicting work.

The adapter implementation and ULC-01 production writer are commissioned. If they are unavailable during a disaster recovery event, preserve the outbox and prepared transaction material; do not replace them with an improvised writer or enable ULC-02 outside the approved HA takeover procedure.

13.10 What is deliberately deferred

Not required to pass the **structural HA POC**:

WAL-G
continuous WAL archiving
PITR
paid object storage
multi-region backup replication
long production retention schedules
production RPO/RT0 certification

These are future deployment hardening, not reasons to remove Patroni, TimescaleDB, OpenBao, or the outbox from the POC.

13.11 Phase 13A pass condition

For the current **non-destructive normal-path commissioning**, Phase 13A passes when:

- the already-validated current chirpstack and lorawan_telemetry logical dumps exist outside the target Droplets;
- the off-host transport archive/hash matches the source transport hash;
- current non-secret HAProxy/PgBouncer/Mosquitto/Valkey/ChirpStack configuration references remain documented;
- when a legacy authoritative source exists, its final migration backup/off-host destination is ready.

Do not repeat source `pg_restore --list` or dump SHA-256 checks when those exact dump files have already passed and have not changed.

Before **Phase 15 destructive/failover testing**, complete the stronger DR gate that was intentionally deferred for speed: isolated `lorawan_telemetry` restore with Timescale objects intact, required `etcd` snapshot/member/config evidence, and any other destructive-recovery prerequisite relevant to the planned fault.

After the streamlined **13A PASS**, continue to [12-gateway-and-device-migration.md](#) only when the Phase 11 normal path is also ready. Do not start fault injection until the stronger pre-Phase-15 DR gate passes.

13.12 Phase 13B final full-stack snapshot

After Node-RED, Grafana, OpenBao, and the Fabric adapters have passed their normal-path setup, repeat the backup collection and add:

```
Node-RED flow export + image/palette/config references
Grafana dashboard/data-source export without credentials
OpenBao recovery material protected off-host
OpenBao Raft snapshot + version/member record
Fabric adapter immutable image/source reference + worker IDs/config reference
Fabric external handoff metadata + last normal confirmed tx reference
public-ingress Reserved IPv4/Droplet/anchor worksheet
public-ingress script/systemd hashes and protected token reference
final schema/migration/outbox state
final service image/config hashes needed to recreate the tested baseline
```

Re-check the database dumps and isolated restore against the final schema. Phase 13B must not rely only on the earlier 13A archives because the application/outbox state has changed.

Phase 13B PASS does not itself authorize fault injection. Next run the healthy evidence-harness dry run in [14-observability-alerting-and-logging.md](#), then the hard gate in [14b-pre-test-commissioning-gate.md](#).

Phase 13A transport archive created PASS - 2026-08-28

The streamlined off-host transport archive was created successfully on `u1c-03` from the already-validated source directory `/home/opsadmin/backups/phase13a-20260827T032756Z`. The resulting archive is `/home/opsadmin/backups/phase13a-20260827T032756Z.tar.gz`, size about 28 KiB, with SHA-256 `e97d50c31252ede1fe55b734b6686f270e92ebecb69a36d637b04fbf726cda1c`. The sidecar `phase13a-20260827T032756Z.tar.gz.sha256` was created successfully. Treat archive creation as PASS. The only remaining fast-path transport gate is to copy the archive to the administration workstation and confirm the destination SHA-256 matches exactly; do not recreate the database dumps or rerun their source validation.

Supporting runbook: 05-etcd-cluster.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

5. Three-Member etcd Cluster

Status: CORE DEPLOYMENT VALIDATED. The Docker Compose configuration, three-member bootstrap, failure corrections, member reachability, leader election, and final 3/3 endpoint status were actually tested on ulc-01, ulc-02, and ulc-03. The later snapshot, planned maintenance, member replacement, quorum-loss recovery, monitoring, and optional TLS-hardening sections are operational guidance and have **not** yet been live-rehearsed unless a later execution-log entry says otherwise.

5.1 Role and current deployment

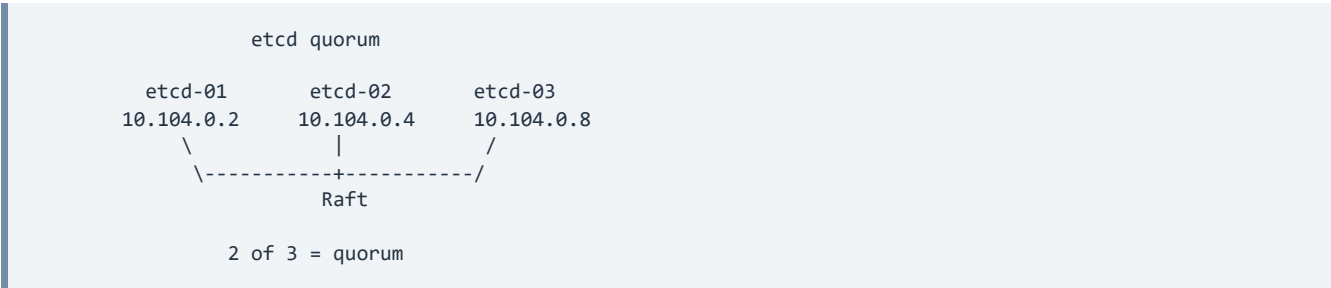
etcd is the distributed coordination store used by Patroni for leader election and PostgreSQL cluster state. It is not the application database and must not be used as a general telemetry key-value store.

The tested cloud POC runs one etcd member on each HA host:

Physical host	Logical role	etcd member	East-west IP	Client	Peer
ulc-01	ha-01	etcd-01	10.104.0.2	2379/tcp	2380/tcp
ulc-02	ha-02	etcd-02	10.104.0.4	2379/tcp	2380/tcp
ulc-03	ha-03	etcd-03	10.104.0.8	2379/tcp	2380/tcp

The validated server image is quay.io/coreos/etcd:v3.5.15. etcd runs with Docker Compose and network_mode: host so it can bind directly to the stable eth1 east-west addresses.

A running process on one host is not enough. A three-member etcd cluster needs a majority of two members to elect a leader and commit changes.



5.2 Network decision proven during deployment

Do not use the 10.15.0.x addresses for HA traffic.

The hosts have additional addresses on eth0:

```
ulc-01 10.15.0.5/16
ulc-02 10.15.0.7/16
ulc-03 10.15.0.6/16
```

Cross-node pings to these addresses failed. `ip neigh show` and `arp -n` showed unresolved or failed neighbors, so the `10.15.0.0/16` path was rejected for cluster traffic.

The `eth1` addresses were then tested:

```
ulc-01 10.104.0.2/20
ulc-02 10.104.0.4/20
ulc-03 10.104.0.8/20
```

Cross-node ICMP succeeded, and TCP peer traffic was proven with `nc` on port 2380. These are therefore the operationally validated east-west addresses for `etcd` and the following HA services.

Why this check comes before `etcd`: a member can have a valid local IP and route while still being unable to reach its peers. `etcd` would then loop through elections and never reach quorum.

The operator is not authorized to change the DigitalOcean Cloud Firewall. Do not change it from this procedure. UFW also remains a separate controlled host-hardening task. The current `etcd` listeners bind only to the `10.104.0.x` east-west addresses plus local loopback for the client API; they must never be changed to `0.0.0.0` on these public hosts.

5.3 Preconditions

Before the first bootstrap, confirm all of the following:

- `ulc-01`, `ulc-02`, and `ulc-03` resolve to the intended east-west addresses through `/etc/hosts`;
- `10.104.0.2`, `10.104.0.4`, and `10.104.0.8` can reach each other;
- TCP 2380 works between hosts;
- Docker Engine and the Compose plugin work on all three hosts;
- Docker host networking exists;
- time is synchronized;
- `/opt/lorawan/etcd/data` is empty on every member for a brand-new cluster;
- member names and IPs are fixed before first startup.

Useful checks:

```
hostname
ip -4 -br address
ip route
ping -c 3 10.104.0.2
ping -c 3 10.104.0.4
ping -c 3 10.104.0.8
docker --version
docker compose version
docker network ls
```

For a new cluster, stop here if a data directory already contains `member/` and its origin is not understood.

5.4 Directory layout

The tested layout is:

```
/opt/lorawan/etcd
├── config
│   └── etcd.yml
├── data
└── docker-compose.yml
```

The live evidence confirms this directory layout and the final data mode correction to 700, but it does not preserve one authoritative original `chown/install` command for every directory. Do not invent historical ownership from the final screenshot.

For a fresh rebuild, first inspect the intended operator/group, then create the layout deliberately. With `opsadmin` as the deployment operator, one reproducible baseline is:

```
sudo install -d -m 755 -o opsadmin -g opsadmin /opt/lorawan/etcd
sudo install -d -m 755 -o opsadmin -g opsadmin /opt/lorawan/etcd/config
sudo install -d -m 700 -o opsadmin -g opsadmin /opt/lorawan/etcd/data
stat -c '%a %U:%G %n' /opt/lorawan/etcd /opt/lorawan/etcd/config /opt/lorawan/etcd/data
```

Treat those ownership commands as the **rebuild baseline**, not a claim about the exact original creation command. The proven security correction is that data must be mode 700.

Why data is mode 700: etcd warned when the data directory was group/world accessible. The database contains Raft state and cluster metadata and does not need general host-user access.

5.5 Docker Compose definition

Create the same Compose file on every node. `sudo tee` is used because shell redirection does not inherit `sudo` privileges.

```
sudo tee /opt/lorawan/etcd/docker-compose.yml >/dev/null <<'EOF'
services:
  etcd:
    image: quay.io/coreos/etcd:v3.5.15
    container_name: etcd
    restart: unless-stopped
    network_mode: host
    command:
      - /usr/local/bin/etcd
      - --config-file=/etc/etcd/etcd.yml
    volumes:
      - /opt/lorawan/etcd/config/etcd.yml:/etc/etcd/etcd.yml:ro
      - /opt/lorawan/etcd/data:/etcd-data
EOF
```

Validate it before startup:

```
cd /opt/lorawan/etcd
docker compose config
```

The output must show the config bind mount targeting `/etc/etcd/etcd.yml`, the data bind mount targeting `/etcd-data`, and `network_mode: host`.

5.6 etcd configuration - u1c-01

Create the file exactly as one etcd configuration document. Do not put a `services:` block here; that belongs only in `docker-compose.yml`.

```
sudo tee /opt/lorawan/etcd/config/etcd.yml >/dev/null <<'EOF'
name: etcd-01

data-dir: /etcd-data

initial-cluster: etcd-01=http://10.104.0.2:2380,etcd-02=http://10.104.0.4:2380,etcd-03=http://10.104.0.8:2380
initial-cluster-state: new
initial-cluster-token: lorawan-etcd-cluster

listen-peer-urls: http://10.104.0.2:2380
initial-advertise-peer-urls: http://10.104.0.2:2380

listen-client-urls: http://10.104.0.2:2379,http://127.0.0.1:2379
advertise-client-urls: http://10.104.0.2:2379
EOF
```

5.7 etcd configuration - u1c-02

```
sudo tee /opt/lorawan/etcd/config/etcd.yml >/dev/null <<'EOF'
name: etcd-02

data-dir: /etcd-data

initial-cluster: etcd-01=http://10.104.0.2:2380,etcd-02=http://10.104.0.4:2380,etcd-03=http://10.104.0.8:2380
initial-cluster-state: new
initial-cluster-token: lorawan-etcd-cluster

listen-peer-urls: http://10.104.0.4:2380
initial-advertise-peer-urls: http://10.104.0.4:2380

listen-client-urls: http://10.104.0.4:2379,http://127.0.0.1:2379
advertise-client-urls: http://10.104.0.4:2379
EOF
```

5.8 etcd configuration - u1c-03

```
sudo tee /opt/lorawan/etcd/config/etcd.yml >/dev/null <<'EOF'
name: etcd-03

data-dir: /etcd-data

initial-cluster: etcd-01=http://10.104.0.2:2380,etcd-02=http://10.104.0.4:2380,etcd-03=http://10.104.0.8:2380
initial-cluster-state: new
initial-cluster-token: lorawan-etcd-cluster

listen-peer-urls: http://10.104.0.8:2380
initial-advertise-peer-urls: http://10.104.0.8:2380

listen-client-urls: http://10.104.0.8:2379,http://127.0.0.1:2379
advertise-client-urls: http://10.104.0.8:2379
EOF
```

5.9 Configuration rules that must match before first startup

On all three nodes, this line must be identical and kept on one physical YAML line:

```
initial-cluster: etcd-01=http://10.104.0.2:2380,etcd-02=http://10.104.0.4:2380,etcd-03=http://10.104.0.8:2380
```

Only these node-specific values change:

```
ulc-01 -> name etcd-01 -> 10.104.0.2
ulc-02 -> name etcd-02 -> 10.104.0.4
ulc-03 -> name etcd-03 -> 10.104.0.8
```

Verify each file:

```
cat /opt/lorawan/etcd/config/etcd.yml
grep -E 'name:|initial-cluster:|listen-peer-urls:|initial-advertise-peer-urls:|listen-client-urls:|advertise-
client-urls:' \
/opt/lorawan/etcd/config/etcd.yml
```

Why the single-line initial-cluster matters here: during the live bootstrap, the folded YAML form produced spaces before later member names in the parsed string. `etcd-02` and `etcd-03` then failed with `couldn't find local name ... in the initial cluster configuration`. The single-line form removes that ambiguity and is the tested form.

Also keep URL-valued settings as strings. In `etcd v3.5.15`, using YAML arrays such as:

```
advertise-client-urls:
- http://10.104.0.2:2379
```

caused startup to fail with:

```
json: cannot unmarshal array into Go struct field configYAML.advertise-client-urls of type string
```

5.10 Clean initial bootstrap

For a brand-new cluster, all three active data directories must be empty before the successful bootstrap.

If a previous failed bootstrap already wrote member state, stop the containers first:

```
cd /opt/lorawan/etcd
docker compose down
```

Before deleting anything, prove all three `etcd` containers are stopped and inspect the exact data state on each node:

```
cd /opt/lorawan/etcd
docker compose ps
sudo find /opt/lorawan/etcd/data -mindepth 1 -maxdepth 2 -printf '%M %U:%G %p\n'
```

Only if this is a **brand-new failed initial bootstrap**, no application depends on this `etcd` state, and a full rebootstrap has been explicitly accepted, remove only the failed `etcd` member state:

```
sudo rm -rf -- /opt/lorawan/etcd/data/member
sudo test ! -e /opt/lorawan/etcd/data/member
sudo chmod 700 /opt/lorawan/etcd/data
sudo stat -c '%A %a %U:%G %n' /opt/lorawan/etcd/data
sudo find /opt/lorawan/etcd/data -mindepth 1 -maxdepth 1 -print
```

Do **not** use a wildcard such as `/opt/lorawan/etcd/data/*` in the reusable procedure. Removing only the known `etcd` member directory avoids deleting unrelated files that may later share the parent path. Once an accepted cluster contains real coordination state, do not use this initial-bootstrap reset at all; use `etcd` membership/snapshot/recovery procedures instead.

Bootstrap the static three-member cluster **sequentially for observability**. Simultaneous startup is not required because every member already has the complete `initial-cluster` list.

1. On `ulc-01`, start `etcd-01` and inspect it:

```
bash cd /opt/lorawan/etcd docker compose up -d docker logs etcd --tail 80
```

Expected: `etcd-01` knows all three members but cannot form quorum by itself. Election/pre-vote retries while the other two peers are down are expected.

2. On `ulc-02`, start `etcd-02` and inspect both nodes. With two of three voters available, the cluster can now reach quorum and elect a leader.

```
bash cd /opt/lorawan/etcd docker compose up -d docker logs etcd --tail 80
```

3. On `ulc-03`, start `etcd-03` and inspect it:

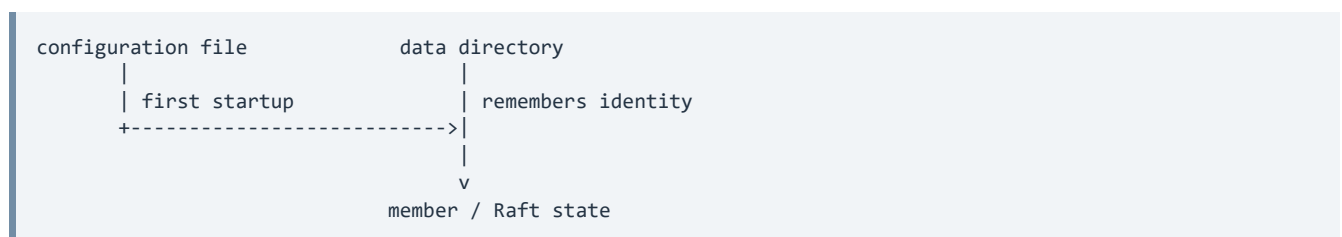
```
bash cd /opt/lorawan/etcd docker compose up -d docker logs etcd --tail 80
```

Expected: the third member joins the already-quorate cluster and restores full three-member redundancy.

Why sequential startup: it lets the operator distinguish the expected one-member no-quorum state from the two-member quorum transition and the final third-member join. Starting all three nearly simultaneously is also valid for a correctly configured static cluster, but it is not a requirement and gives less useful bootstrap evidence.

5.11 Why a failed bootstrap must be cleaned before retrying

`etcd` is stateful. The data directory contains the member ID, cluster ID, Raft log, snapshots, and peer membership.



Changing `etcd.yml` after a failed first start does not rewrite the old member identity. During the live setup this produced messages such as `server has been already initialized` and continued bootstrap failures.

Therefore, during **initial deployment only**, a failed bootstrap is recovered by stopping all three members, correcting every configuration file, clearing all three failed data directories, and then bootstrapping again.

After the cluster is accepted and contains real state, **never use this cleanup procedure**. Normal maintenance and member replacement use the member APIs and snapshots instead.

5.12 Troubleshooting notes from the live deployment

Docker Compose YAML accidentally used as etcd YAML

The first `/opt/lorawan/etcd/config/etcd.yml` contained a `Docker services:` definition. `etcd` therefore did not receive the intended member configuration and started with default/localhost values during the early attempt.

Keep the responsibilities separate:

<code>docker-compose.yml</code>	-> Docker container lifecycle
<code>config/etcd.yml</code>	-> etcd daemon configuration

data/

-> etcd persistent Raft state

advertise-client-urls array error

Symptom:

```
failed to verify flags  
json: cannot unmarshal array into Go struct field configYAML.advertise-client-urls of type string
```

Fix: use comma-separated string values, not YAML arrays, for the URL fields in this pinned version.

Member name not found

Symptom:

```
couldn't find local name "etcd-02" in the initial cluster configuration
```

or the same error for etcd-03.

Fix: make `initial-cluster` a single exact line with no spaces after commas, verify the local name: matches exactly, then clean failed bootstrap data before retrying.

Peer connection refused

Symptom:

```
dial tcp 10.104.0.4:2380: connect: connection refused
```

This means the network path reached the target host but no healthy etcd peer was listening yet. During bootstrap, check the target container logs instead of immediately treating this as a VPC failure.

Repeated elections / request timeout

A single member of the intended three-member cluster cannot get a majority. Logs can show repeated starting a new election, `MsgPreVote`, or publish timeouts until at least one other configured peer becomes available.

5.13 Health and quorum validation

From an administrative host with a compatible `etcdctl`, run:

```
ETCDCTL_API=3 etcdctl \  
--endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \  
endpoint status \  
--write-out=table
```

Also record:

```
ETCDCTL_API=3 etcdctl \  
--endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \  
endpoint health  
  
ETCDCTL_API=3 etcdctl \  
--endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \  
member list \  
--write-out=table
```

The successful deployment on 2026-08-21 showed all three endpoints on etcd 3.5.15, one leader, two followers, no learners, matching Raft term/index progression, and no endpoint errors. At that checkpoint etcd-02 (10.104.0.4) was the leader. The leader is allowed to change later; health depends on quorum, not on a particular node remaining leader.

Healthy acceptance requires:

- exactly three expected members;
- all three client endpoints respond;
- exactly one leader;
- two followers;
- no unexpected learner;
- no endpoint errors;
- Raft term/index values progress normally.

5.14 Current transport security boundary

The tested POC currently uses HTTP for etcd client and peer transport **only on the validated 10.104.0.0/20 east-west network**. This is the configuration that was actually bootstrapped and tested.

This is not the same as application-level authentication or transport encryption. Do not expose 2379 or 2380 through a public listener, and do not claim that an unverified DigitalOcean firewall protects them.

The current HTTP-only transport is an **accepted POC checkpoint, not a production transport-security sign-off**. Before this environment is called production-ready, resolve this explicitly: either deploy and validate etcd transport TLS (cluster CA, unique member certificates/SANs, Patroni client trust, controlled rollout, rollback) or record an approved security exception with compensating controls. Do not silently carry the POC HTTP setting into production, and do not mix `http://` and `https://` values during a bootstrap attempt.

Not yet rehearsed: sections 5.15 through 5.19 are operational guidance. No execution-log entry currently proves snapshot restore, member replacement, quorum-loss recovery, or the monitoring procedure has been exercised.

5.15 Snapshots

Create a snapshot from a healthy endpoint and keep it outside the etcd data directory:

```
install -d -m 700 ~/backups/etcd
umask 077
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379 \
  snapshot save ~/backups/etcd/etcd-$(date +%Y%m%d-%H%M%S).db
```

Validate with a compatible etcdctl when available and record a checksum:

```
etcdctl snapshot status ~/backups/etcd/<SNAPSHOT_FILE> --write-out=table
sha256sum ~/backups/etcd/<SNAPSHOT_FILE>
```

Copy accepted snapshots off-host. A snapshot contains coordination state and should be handled as protected infrastructure data.

5.16 Normal maintenance

After the successful bootstrap, do not empty `/opt/lorawan/etcd/data` during a normal restart or upgrade.

Before stopping one member:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health
```

Proceed only when the other two members are healthy. Maintain one member at a time and prove quorum again before touching the next member.

5.17 Failed member replacement

Do not copy another live member's data directory.

Safe outline:

1. prove the failed member cannot unexpectedly return with its old data;
2. snapshot the healthy cluster;
3. remove the failed member with `etcdctl member remove <MEMBER_ID>`;
4. add the replacement with `etcdctl member add` and its exact peer URL;
5. use the member-add output to construct the replacement configuration with `initial-cluster-state: existing`;
6. start the replacement with an empty data directory;
7. verify member list and endpoint health before Patroni depends on it.

Do not remove a member when quorum is already lost without following the version-specific disaster-recovery procedure.

5.18 Quorum loss

Three members tolerate one unavailable member. Two unavailable members mean no majority.

When quorum is lost:

- freeze unrelated changes;
- preserve member data and logs;
- determine whether members are down or network-partitioned;
- restore connectivity first when possible;
- use a validated snapshot only when normal quorum recovery is impossible;
- verify etcd and Patroni state before allowing PostgreSQL writes.

5.19 Monitoring

Collect and alert on:

- leader changes;
- endpoint health;

- proposal failures;
- fsync/backend commit latency;
- database size/quota use;
- peer RTT;
- expected member count;
- snapshot age.

5.20 Final checks

- etcd-01, etcd-02, and etcd-03 form one three-member cluster.
- All three use the 10.104.0.x east-west addresses.
- 2379 and 2380 are not deliberately bound to public addresses.
- One leader and two followers are visible in `endpoint status`.
- No learner or endpoint error remains.
- The initial bootstrap mistakes and recovery sequence are recorded in the execution log.
- Future restarts preserve the accepted data directories.

Next standby phase when work resumes: [06-spilo-patroni-postgresql-cluster.md](#).

Canonical source: `deployment/technology-guides/06-etcd.md`. For any mutable production-status statement, check the dated baseline and verified live state.